

nity GC 系列教程第五篇：高级 GC 内核

已于 2025-07-25 10:51:19 修改 · 1.5k 阅读 · 21 点赞 · 9 收藏 · CC 4.0 BY-SA版权

标签：[#unity](#) [#游戏引擎](#) [#架构](#) [#c#](#)

ity GC 系列教程第一篇：GC 基础概念与工作原理

ity GC 系列教程第二篇：Unity 中常见的 GC Alloc 场景与分析工具

ity GC 系列教程第三篇：GC Alloc 优化技巧与实践（上）

ity GC 系列教程第四篇：GC Alloc 优化技巧与实践（下）与 GC 调优

ity GC 系列教程第五篇：高级 GC 内核

欢迎来到 [Unity](#) GC 系列教程的最后一篇！在前四篇文章中，我们从 GC 的基础概念一路走来，掌握了如何识别和优化常见的 GC Alloc 场景，并了解了 Incremental GC 如何改善游戏 [流畅性](#)。现在，是时候将我们的视野拓宽到更高的层次了。

我们将探讨一些更高级的内存管理和高性能计算技术，如 **C# Job System**、**Burst Compiler** 和 **Native Container**。这些技术不仅能帮助我们从根本上避免 GC Alloc 的性能开销，还能解锁并行计算的强大能力，为 Unity 游戏的性能带来质的飞跃。最后，我们还会展望未来，看看随着 .NET 和 [Unity 引擎](#) 的发展，GC 优化将走向何方。

ScriptableObject 与静态数据

在讨论高性能计算之前，我们先回到 Unity 相对更传统但同样重要的优化点：**ScriptableObject**。虽然它本身不直接涉及复杂的 GC [算法](#)，但合理利用它可以帮助开发者更好地管理数据，从而间接减少运行时内存分配和 GC 压力。

1.1 ScriptableObject：数据存储与共享的利器

ScriptableObject 是一种特殊类型的 Unity 对象，它不与任何 [GameObject](#) 关联，可以独立于场景存在。它的主要用途是作为**数据容器**。

为什么 ScriptableObject 有助于 GC 优化？

1. **数据与逻辑分离**：ScriptableObject 允许你将数据（如游戏配置、技能数据、敌人属性等）与行为逻辑（MonoBehaviour）分离。这些数据可以在编辑器中创建和序列化，并在运行时以引用的方式加载。

2. **减少运行时实例化**：

- 如果你把所有配置都写在 MonoBehaviour 中，每次场景加载或 GameObject 实例化时，这些数据都会随之被复制一份到内存中。
- 而将数据存储在 ScriptableObject 中，你只需在运行时加载并引用一个 ScriptableObject 实例。无论有多少个 GameObject 需要访问这些数据，它们都共享一个 ScriptableObject 实例，**避免了数据的重复拷贝和重复的 GC Alloc。**

3. **编辑器友好**：在编辑器中，你可以像创建 Prefab 一样创建 ScriptableObject 资产，并用 Inspector 编辑其属性，无需硬编码。

列：

设你有一个敌人类型，每个敌人都有固定的生命值、攻击力、移动速度等属性。

代码示例（硬编码或 MonoBehaviour 重复数据）：

AI运行代码复制

```
1 // Enemy.cs
2 public class Enemy : MonoBehaviour
3 {
4     public int health = 100;
5     public float speed = 5f;
6     public string enemyName = "Goblin";
7     // ...
8 }
9
```

展开

代码示例（使用 ScriptableObject）：

AI运行代码 复制

```
1 using UnityEngine;
2
3 // 1. 定义一个 ScriptableObject 来存储敌人数据
4 [CreateAssetMenu(fileName = "NewEnemyData", menuName = "Game Data/Enemy Data")]
5 public class EnemyData : ScriptableObject
6 {
7     public string enemyName = "Default Enemy";
8     public int maxHealth = 100;
9     public float moveSpeed = 5f;
```

展开

过这种方式，你可以在项目中创建多个 `EnemyData` 资产（如“GoblinData”，“OrcData”），然后让不同的 `EnemyOptimized` Prefab 引用这些共享的 `EnemyData` 实例。这样，这些数据只在内存中存在一份，被多个 `GameObject` 共享，减少了内存占用，也避免了在运行时因数据重复实例化而产生的 GC Alloc。

1.2 静态数据与 GC

静态变量（`static` 关键字修饰的变量）也是一种在程序生命周期内只存在一份的数据。它们是 GC 根的一种，因此除非程序结束，否则它们所引用的对象不会被 GC 回收。

特点：

全局可访问：方便在任何地方访问。

只初始化一次：在程序启动时初始化，避免重复创建。

应用场景：

常量或配置数据：如 `static readonly` 字段。

单例模式：如果你的单例不需要绑定到 `GameObject`，可以考虑纯 C# 静态类。

缓存：例如，我们在第三篇中使用的 `static readonly WaitForSeconds` 对象。

注意事项：

生命周期：静态变量的生命周期与应用程序的生命周期相同。如果静态变量引用了大量数据或复杂对象，它们将一直占用内存，直到应用程序关闭。

内存泄漏风险：如果静态事件 (`static event`) 没有正确地取消订阅，或者静态集合 (`static List`) 不断添加元素而不清理，很容易导致内存泄漏，因为这些对永远不会被 GC 回收。

C#

AI运行代码 复制

```
1 // 潜在的静态内存泄漏
2 public static class EventManager
3 {
4     public static event Action OnSomethingHappened;
5     public static List<object> GlobalCache = new List<object>(); // 如果不 Clear，会持续增长
6 }
7
8 public class LeakyComponent : MonoBehaviour
9 {
```

展开 ▼

解决方案：

- 对于静态事件，使用**弱事件模式 (Weak Event Pattern)** 或确保订阅者在销毁时正确地取消订阅。
- 对于静态集合，确保在适当的时候调用 `Clear()` 方法。

! IL2CPP 对 GC 的影响

ity 的一个重要特性是它支持将 C# 代码编译为 C++ 代码，这就是 **IL2CPP**。IL2CPP 是 Unity 的后端编译技术，它将你的 .NET 中间语言 (IL) 代码转换为 C++ [源代码](#)，然后使用 C++ 编译器编译成原生机器码。

!.1 IL2CPP 如何影响 GC 行为?

IL2CPP 本身并不会改变 C# 的 GC 机制，Unity 仍然使用 Mono 的垃圾回收器。然而，IL2CPP 的编译过程可以带来以下影响：

- 1. **性能提升**：将 C# 编译为 C++ 原生代码，可以获得更接近 C++ 的执行效率。这包括更快的指令执行速度，更有效的内存访问模式，从而降低整体的 CPU 消耗，包括 GC 在内的所有操作。虽然 GC 的逻辑和算法没变，但执行效率更高，理论上相同的工作量可以在更短的时间内完成。
- 2. **AOT (Ahead-of-Time) 编译**：IL2CPP 是一种 AOT 编译。这意味着所有代码都在发布前被编译为机器码，而不是像 JIT (Just-In-Time) 编译那样在运行时进行编译。AOT 编译消除了 JIT 编译器在运行时产生的开销，也避免了某些 JIT 编译可能导致的 GC Alloc。
- 3. **更优化的内存布局**：IL2CPP 在生成 C++ 代码时，可能会对数据结构进行一些优化，从而在底层更高效地管理内存。这间接有助于 GC 的效率，因为更紧凑或更有规律的内存布局可能有助于 GC 遍历和整理。
- 4. **调试复杂性**：虽然性能提升显著，但 IL2CPP 编译后的代码调试起来会更复杂，因为你不能直接在 C# 源代码中设置断点来调试生成的 C++ 代码。Profiler 仍然是你的主要工具。

总结：IL2CPP 并没有改变 GC 的本质（仍然是托管内存和 Mono GC），但通过提高代码执行效率和原生化，可以使得 GC 过程更快、更流畅。对于性能敏感的游戏 **使用 IL2CPP 后端是强烈推荐的**。

! C# Job System 与 Burst Compiler

现在，我们进入 Unity 高性能计算的核心领域。**C# Job System** 和 **Burst Compiler** 是 Unity 推出的两大重要技术，它们能够从根本上解决某些场景下的 GC Alloc 问题并极大地提升代码的执行速度。

!.1 C# Job System：多线程与数据导向设计

Job System 是一套基于 Unity 原生多线程能力的高性能编程框架。它允许你编写多线程代码，将计算密集型任务从主线程（渲染、UI、输入）中卸载到其他 CPU 核心上并行执行。

什么 Job System 有助于减少 GC Alloc?

- 1. **值类型优先 (Value Type First):** Job System 强调使用 **值类型 (struct)** 来存储数据。Job 的数据通常是以 **struct** 形式定义的，并通过 **NativeArray** 等 **原生容器 (Native Container)** 传递。 **struct** 在栈上分配或直接嵌入到其他数据结构中，不会产生 GC Alloc。
- 2. **数据扁平化 (Data Layout Optimization):** Job System 鼓励 **数据导向设计 (Data-Oriented Design, DOD)**。这意味着你需要将数据组织成紧凑、连续的数组形式，而不是分散的对象图。这种扁平化的数据结构非常适合 CPU 的缓存，并且消除了传统面向对象编程中常见的引用链，从而减少了对 GC 的依赖。
- 3. **避免托管对象引用:** Job System 严格限制在 Job 中对托管对象 (**class** 实例) 的直接引用。如果你需要在 Job 中访问托管数据，通常需要通过 **NativeArray** 拷贝一份非托管数据，或者使用一些特殊机制 (如 `[Unity.Collections.LowLevel.Unsafe.NoAlias]` 和 `[Unity.Collections.LowLevel.Unsafe.ReadOnly]`) 这种限制强制开发者使用更底层的、无 GC Alloc 的内存管理方式。
- 4. **并行计算:** 虽然不直接减少 GC Alloc，但并行执行任务可以显著缩短总的计算时间。这意味着即使某些 Job 仍然会处理一些托管数据 (通过间接方式)，但由于算效率的提升，主线程的帧预算可以更充裕，GC 暂停的影响会更小。

Job System 的核心组件:

- 1. **IJob 接口:** 定义一个 Job 任务。
- 2. **JobHandle:** 表示一个 Job 的执行状态，用于管理 Job 的依赖和完成。
- 3. **Schedule() 方法:** 将 Job 调度到工作线程执行。
- 4. **Complete() 方法:** 等待 Job 完成并获取结果。

单示例 (模拟粒子更新) :

AI运行代码复制

```
1 using Unity.Jobs;
2 using Unity.Collections;
```

```
3 using UnityEngine;
4
5 // 定义一个 Job: 更新粒子位置
6 public struct ParticleUpdateJob : IJobParallelFor
7 {
8     // NativeArray 是值类型，并且在非托管内存中分配
9     public NativeArray<Vector3> positions;
```

[展开](#) 

这个例子中，`ParticleUpdateJob` 和 `NativeArray<Vector3>` 都是值类型，且 `NativeArray` 在非托管内存中分配。整个粒子更新过程，从数据存储到 Job 执行基本避免了 GC Alloc，并且能利用多核 CPU 并行加速。

1.2 Burst Compiler: 为 Job 提速

Burst Compiler 是一个 **高性能的、优化的 JIT (Just-In-Time) 编译器**，它将你用 C# 编写的 Job 代码（`IJob` 接口的实现）编译成高度优化的机器码。

什么 Burst Compiler 有助于 GC 优化？

- 1. **极致的性能优化**：Burst Compiler 会对你的 Job 代码进行大量的底层优化，包括 SIMD (Single Instruction, Multiple Data) 指令集优化、缓存优化、死代码消除等。这使得 Job 的执行速度比普通的 C# 代码快很多倍。
- 2. **减少执行时间 = 减少 GC 暂停几率**：虽然 Burst Compiler 本身不直接减少 GC Alloc，但它通过极大地加速 Job 的执行，减少了 CPU 在计算任务上的耗时。这意味着 CPU 有更多空闲时间用于其他任务，包括 GC 的短暂停顿。如果 GC 的工作能够迅速完成，它对帧率的影响就会更小。
- 3. **与 Job System 协同**：Burst Compiler 专为 Job System 设计，它能够将 Job System 结构体中的代码编译成高度优化的原生代码。两者结合，可以实现前所未有的性能。

用 Burst Compiler：

通常只需要在 Unity 的 Package Manager 中安装“Burst”包。一旦安装，所有符合 Burst 编译条件的 Job (实现了 `IJob` 等接口的 struct) 都会自动被 Burst Compiler

列：

38 ParticleUpdateJob 结构体，只要安装了 Burst 包，就会自动被 Burst Compiler 优化。你可以在 Execute 方法上添加 [BurstCompile] 特性来显式指示 Burst

39 er 编译它，尽管这通常是可选的。

40

41

42

AI运行代码

复制

```
1 using Unity.Jobs;
2 using Unity.Collections;
3 using Unity.Burst; // 引入 Burst 命名空间
4 using UnityEngine;
5
6 [BurstCompile] // 显式指示 Burst Compiler 编译这个 Job
7 public struct ParticleUpdateJob : IJobParallelFor
8 {
9     public NativeArray<Vector3> positions;
```

展开 ▼

55

56 **C# Job System 和 Burst Compiler 是 Unity 游戏中实现极致性能，并从根本上减少 GC Alloc 的强大组合。** 它们鼓励数据导向设计，将内存管理从 GC

57 更底层的原生容器，并通过并行和高度优化的编译来加速计算。

58

59

60 **Native Container：非托管内存的管理者**

61 **Native Container** 是 Unity 提供的一组 C# 结构体（如 `NativeArray<T>`，`NativeList<T>`，`NativeHashMap<TKey, TValue>` 等），它们允许你在 **非托管内存**

62 **(Unmanaged Memory)** 中分配和管理数据。

63

64 **Native Container 如此重要？**

65

66 **避免 GC Alloc：**这是最核心的原因。`NativeArray` 等容器在创建时直接向操作系统申请非托管内存，而不是在托管堆上分配。这意味着它们不受 GC 的管

67 理，因此不会产生 GC Alloc，也不会导致 GC 暂停。

68

69 **数据局部性 (Data Locality)：**Native Container 通常将数据存储在连续的内存块中。这对于 CPU 缓存非常友好，可以显著提高数据访问速度。

- 1. **与 Job System 无缝集成**：Native Container 设计用于 Job System。它们可以安全地在多个 Job 之间传递和访问，而不会引发数据竞争问题（通过 Job System 依赖管理和读写权限控制）。
- 2. **手动生命周期管理**：由于它们是非托管内存，你需要手动管理它们的生命周期。这意味着你必须显式地调用 `Dispose()` 方法来释放内存。如果忘记 `Dispose()` 就会导致非托管内存泄漏。

见的 Native Container:

NativeArray<T>：非托管内存中的数组。最基础和常用的 Native Container。

NativeList<T>：非托管内存中的动态列表，类似于 `List<T>`。

NativeHashMap<TKey, TValue>：非托管内存中的哈希表，类似于 `Dictionary<TKey, TValue>`。

NativeQueue<T>：非托管内存中的队列。

NativeStream：用于 Job 之间高效传递数据的结构。

allocator 参数:

创建 Native Container 时，你需要指定一个 Allocator 类型，它决定了内存的分配和释放方式：

Allocator.Temp：用于非常短期的分配，例如在单个 Job 内部。它分配的内存会在当前帧结束时自动释放（或者在 `Complete()` 后）。这是最快的分配方式，生命周期最短。

Allocator.TempJob：用于 Job 中，但生命周期可以跨越 Job 执行。通常用于 Job 的输入和输出数据。内存会在 Job 完成并 `Complete()` 后释放。比 `Temp` 慢比 `Persistent` 快。

Allocator.Persistent：用于长期存在的分配，例如整个游戏生命周期的数据。这些内存必须手动调用 `Dispose()` 释放。这是最慢的分配方式，但生命周期最长。

要提示：

必须 `Dispose()`：如果你使用了 `Allocator.Persistent` 或 `Allocator.TempJob`，切记在不再需要这些容器时调用它们的 `Dispose()` 方法。通常在 `OnDestroy()` 或 `OnDisable()` 中执行。

内存安全：Unity 对 Native Container 做了严格的安全检查，例如，如果你尝试在 Job 执行期间访问已经 `Dispose` 的 Native Container，会立即报错。这有助于避免内存损坏。

列：在 `ParticleSystemController` 示例中我们已经使用了 `NativeArray<Vector3>` 和 `Allocator.Persistent`，并在 `OnDestroy` 中调用了 `Dispose()`。

Addressables 与内存管理

Addressables Asset System 是 Unity 的一个强大的资产管理系统，旨在简化资产加载、卸载和管理复杂性，特别是针对**远程资产和动态加载**。它与 GC 优化息息相关，因为不当的资产加载和卸载会导致内存泄漏和 GC 峰值。

1.1 Addressables 如何帮助管理 Asset Bundle 的生命周期？

传统的 `Resources.Load()` 或手动 `AssetBundle` 管理中，你很容易遇到以下问题：

内存泄漏：忘记 `UnloadAssetBundle(true)` 或 `Resources.UnloadUnusedAssets()`，导致卸载不彻底。

GC 峰值：在不恰当的时机加载/卸载大量资产，导致 GC 压力骤增。

重复加载：多个地方加载同一资产，导致内存中存在多份副本。

Addressables 通过以下方式解决这些问题并协同 GC 优化：

引用计数 (Reference Counting)：Addressables 内部维护一个资产的引用计数。只有当一个资产的所有引用都释放时，Addressables 才会将其标记为可卸载。当你调用 `Addressables.LoadAssetAsync()` 时，引用计数增加；当你调用 `Addressables.Release()` 或 `ReleaseInstance()` 时，引用计数减少。

自动卸载：当资产的引用计数降为零时，Addressables 会自动将其从内存中卸载。这大大简化了开发者的内存管理负担，减少了手动卸载可能导致的错误。

- 1. **异步加载与卸载**：Addressables 的所有加载和卸载操作都是异步的，这意味着它们不会阻塞主线程，从而避免卡顿。你可以在加载屏幕期间进行这些操作。
- 2. **资源合并与重复数据消除**：Addressables 构建系统能够分析资产依赖，并将共享资产合并到同一个 Asset Bundle 中，避免重复打包和内存占用。

用 Addressables 减少 GC Alloc 的实践：

使用 **LoadAssetAsync<T>()** 加载资产：而不是 **Resources.Load()** 。

使用 **Release()** 释放资产：当不再需要资产时，**务必调用 Addressables.Release()**。这是 Addressables 内存管理的核心。

C#

AI运行代码 复制

```
1 using UnityEngine;
2 using UnityEngine.AddressableAssets;
3 using UnityEngine.ResourceManagement.AsyncOperations;
4 using System.Collections;
5
6 public class AddressablesExample : MonoBehaviour
7 {
8     public AssetReferenceGameObject playerPrefabRef; // 在 Inspector 中拖拽分配
9     private GameObject _spawnedPlayer;
```

展开

Addressables.InstantiateAsync() 与对象池：InstantiateAsync 方法可以异步实例化资产。如果你想将 Addressables 加载的 Prefab 进行对象池化，你需从 InstantiateAsync 获得的 GameObject 实例中提取数据并将其放入你的对象池，或者直接将 Addressables.InstantiateAsync 的结果归还到池中（如果的池设计支持）。

- **重要**：当你使用 Addressables.InstantiateAsync() 实例化一个 GameObject 时，你需要在不再需要它时调用 Addressables.ReleaseInstance(GameObject instance) 来释放它，而不是简单的 Destroy()。ReleaseInstance 会同时处理对象的销毁和资产引用计的减少。

总结：Addressables 能够系统性地管理资产加载和卸载，并通过引用计数机制避免内存泄漏。虽然它本身不是一个 GC 优化技术，但它与 GC 协同工作，让你能够更有效地控制游戏中的内存占用和资源生命周期，从而间接减少因资源管理不当导致的 GC 压力。

5 未来展望：.NET GC 发展与 Unity 的演进

随着 .NET GC 技术和 .NET Runtime 都在不断发展。了解这些趋势有助于我们更好地规划未来的优化策略。

5.1 .NET 中的 GC 改进

Unity 目前（截至 Unity 2024.x）使用的 .NET 版本（主要是 .NET Standard 2.1，或 Unity 2022/2023 中的 .NET 6/7）已经包含了许多 GC 方面的改进：

更智能的 GC 算法：.NET 的 GC (特别是 .NET Core / .NET 5+) 引入了更先进的算法和启发式策略，例如 **Background GC** (在后台线程执行部分 GC 工作，减少 STW 时间) 和 **Concurrent GC** (并发 GC，与应用程序线程同时运行，进一步减少 STW 时间)。

分代 GC 的优化：对新生代和老年代的收集过程进行了持续优化，提高了效率。

内存段的管理优化：更有效地管理内存段，减少碎片。

LOH (Large Object Heap) 优化：针对大对象的内存分配和回收进行了改进，因为大对象通常直接进入老年代，其回收成本较高。

随着 Unity 持续更新其 .NET Runtime 版本（例如，未来全面升级到 .NET 8+），我们有望在 Unity 中看到更多这些底层 GC 优化的好处。这意味着，即使我们不主动进行大量 GC Alloc 优化，底层的 GC 也会变得越来越智能和高效。

5.2 Unity 未来可能推出的 GC 优化方案

除了跟随 .NET Runtime 的发展，Unity 自身也在积极探索更深层次的内存管理和 GC 优化：

1. DOTS (Data-Oriented Technology Stack) 的成熟：C# Job System、Burst Compiler 和 Native Container 是 DOTS 的核心组成部分。随着 DOTS 的进一步成熟和普及，更多开发者将采用数据导向设计，从根本上减少对托管堆的依赖。未来的 Unity 版本可能会有更多 API 以 DOTS 风格提供，鼓励无 GC Alloc 的编程模式。

2. Value Types (值类型) 的进一步推广：Unity 可能会继续推广使用值类型，甚至在某些场景下提供工具或模式，将传统的引用类型数据转换为值类型数据，从而减轻 GC 压力。

- 1. **更好的 Profiler 工具**：Unity 可能会不断改进其 Profiler 和 Memory Profiler，提供更精细的 GC 分析功能，帮助开发者更容易地识别和解决内存问题。例如，更详细的 GC 日志、内存分配的来源追踪工具等。
- 2. **自定义 GC 策略**：虽然目前 Unity 不支持完全自定义 GC 策略，但未来可能会提供更高级的选项，允许开发者在特定平台或场景下微调 GC 行为，以适应更极端的性能需求。
- 3. **Memory Management Frameworks**：Unity 可能会推出或集成更高级的内存管理框架，进一步简化原生内存的使用，降低开发者自己管理 `Dispose` 的复杂度。

1.3 其他更底层的内存管理技术 (简述)

自定义内存分配器 (Custom Allocators)：在非常底层的 C++ 或高性能库中，有时会使用自定义内存分配器，完全绕过操作系统默认的内存分配机制，以实现更优化的分配和回收策略。在 Unity 的 C# 层，这通常通过 Native Container 来间接实现。

Memory Pooling at Native Level：除了 C# 层面的对象池，游戏引擎在原生层面也会进行大量的内存池化，例如渲染数据、物理缓存等。

结

比，我们的 Unity GC 系列教程圆满结束。在第五篇中，我们探讨了：

ScriptableObject 和静态数据 如何作为数据管理工具，间接减少运行时内存分配。

IL2CPP 作为后端编译技术，如何通过原生代码执行效率的提升来间接优化 GC。

C# Job System 和 Burst Compiler：这是 Unity 高性能计算的基石，它们通过数据导向设计、值类型优先和原生容器，从根本上解决 GC Alloc 问题，并利用多 CPU 提升性能。

Native Container：它们是非托管内存的管理者，允许我们彻底避免 GC Alloc，但需要手动管理生命周期。

Addressables Asset System：它简化了资源加载和卸载的复杂性，通过引用计数机制有效防止内存泄漏，从而间接减轻 GC 压力。

最后，我们展望了 **.NET GC 的未来发展** 和 **Unity 引擎在 GC 优化方面的演进**。

整个系列，我们从 GC 的基本概念到高级优化技术，一步步深入。最重要的是，我们强调了以下核心原则：

- 1. **理解原理**：知其然，知其所以然。
- 2. **测量先行**：在优化前，永远使用 **Unity Profiler** 定位真正的瓶颈。
- 3. **优先减少 GC Alloc**：这是 GC 优化的治本之道。
- 4. **合理利用工具和技术**：根据项目需求和性能瓶颈，选择合适的优化方法，从传统的代码优化到现代的高性能计算技术。
- 5. **持续迭代**：性能优化是一个永无止境的过程。

希望这个系列教程能为你提供一个全面、深入的 Unity GC 优化指南，帮助你在游戏开发中写出更流畅、更高效的代码！如果你有任何疑问或想进一步探讨的话题，欢迎在评论区提出。

[Unity GC 系列教程第一篇：GC 基础概念与工作原理](#)

[Unity GC 系列教程第二篇：Unity 中常见的 GC Alloc 场景与分析工具](#)

[Unity GC 系列教程第三篇：GC Alloc 优化技巧与实践（上）](#)

[Unity GC 系列教程第四篇：GC Alloc 优化技巧与实践（下）与 GC 调优](#)

[Unity GC 系列教程第五篇：高级 GC 内核](#)

[关于我们](#) [招贤纳士](#) [商务合作](#) [寻求报道](#) ☎ 400-660-0108 ✉ kefu@csdn.net 🗣 在线客服 工作时间 8:30-22:00

公安备案号11010502030143 京ICP备19004658号 京网文〔2020〕1039-165号 经营性网站备案信息 北京互联网违法和不良信息举报中心

家长监护 网络110报警服务 中国互联网举报中心 Chrome商店下载 账号管理规范 版权与免责声明 版权申诉 出版物许可证 营业执照

©1999-2025北京创新乐知网络技术有限公司